

coursework_02

March 3, 2023

1 Coursework 2: Image segmentation

In this coursework you will develop and train a convolutional neural network for brain tumour image segmentation. Please read both the text and the code in this notebook to get an idea what you are expected to implement. Pay attention to the missing code blocks that look like this:

```
### Insert your code ###  
...  
### End of your code ###
```

1.1 What to do?

- Complete and run the code using `jupyter-lab` or `jupyter-notebook` to get the results.
- Export (File | Save and Export Notebook As...) the notebook as a PDF file, which contains your code, results and answers, and upload the PDF file onto [Scientia](#).
- Instead of clicking the Export button, you can also run the following command instead:
`jupyter nbconvert coursework.ipynb --to pdf`
- If Jupyter complains about some problems in exporting, it is likely that `pandoc` (<https://pandoc.org/installing.html>) or `latex` is not installed, or their paths have not been included. You can install the relevant libraries and retry.
- If Jupyter-lab does not work for you at the end, you can use Google Colab to write the code and export the PDF file.

1.2 Dependencies

You need to install Jupyter-Lab (https://jupyterlab.readthedocs.io/en/stable/getting_started/installation.html) and other libraries used in this coursework, such as by running the command: `pip3 install [package_name]`

1.3 GPU resource

The coursework is developed to be able to run on CPU, as all images have been pre-processed to be 2D and of a smaller size, compared to original 3D volumes.

However, to save training time, you may want to use GPU. In that case, you can run this notebook on Google Colab. On Google Colab, go to the menu, Runtime - Change runtime type, and select **GPU** as the hardware accelerator. At the end, please still export everything and submit as a PDF file on Scientia.

```
[1]: # Import libraries
# These libraries should be sufficient for this tutorial.
# However, if any other library is needed, please install by yourself.
import tarfile
import imageio
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torch.utils.data import Dataset
import numpy as np
import time
import os
import random
import matplotlib.pyplot as plt
from matplotlib import colors
```

1.4 1. Download and visualise the imaging dataset.

The dataset is curated from the brain imaging dataset in [Medical Decathlon Challenge](#). To save the storage and reduce the computational cost for this tutorial, we extract 2D image slices from T1-Gd contrast enhanced 3D brain volumes and downsample the images.

The dataset consists of a training set and a test set. Each image is of dimension 120 x 120, with a corresponding label map of the same dimension. There are four number of classes in the label map:

- 0: background
- 1: edema
- 2: non-enhancing tumour
- 3: enhancing tumour

```
[2]: # Download the dataset
!wget https://www.dropbox.com/s/zmytk2yu284af6t/Task01_BrainTumour_2D.tar.gz

# Unzip the '.tar.gz' file to the current directory
datafile = tarfile.open('Task01_BrainTumour_2D.tar.gz')
datafile.extractall()
datafile.close()

### My code ###

main_path = "./Task01_BrainTumour_2D/"
training_images_path = main_path + "training_images/"
training_labels_path = main_path + "training_labels/"
test_images_path = main_path + "test_images/"
test_labels_path = main_path + "test_labels/"
```

```

image_color_map = "gray"
label_color_map = colors.ListedColormap(['black', 'green', 'blue', 'red'])

### End of my code ###

```

--2023-03-03 11:20:07--

https://www.dropbox.com/s/zmytk2yu284af6t/Task01_BrainTumour_2D.tar.gz

Resolving www.dropbox.com (www.dropbox.com)... 162.125.4.18,

2620:100:6019:18::a27d:412

Connecting to www.dropbox.com (www.dropbox.com)|162.125.4.18|:443... connected.

HTTP request sent, awaiting response... 302 Found

Location: /s/raw/zmytk2yu284af6t/Task01_BrainTumour_2D.tar.gz [following]

--2023-03-03 11:20:08--

https://www.dropbox.com/s/raw/zmytk2yu284af6t/Task01_BrainTumour_2D.tar.gz

Reusing existing connection to www.dropbox.com:443.

HTTP request sent, awaiting response... 302 Found

Location: https://uce3a5cdf30abe772e14002d1d64.dl.dropboxusercontent.com/cd/0/inline/B3i5iKbQi6aU9tcS8iIlC10EzPkychJ7mXM5LhvsLRpIhgsvG-GcdOMSTiQyegGdTHJp9MYWNSCW4gFf10eqy1aJlFmlja30sJHom4YmBpxyGi7hD2u1_COSjuwfv7UxexRAam9enmoC_pOETn11f2cPbZ0YMNIFNI1UV_wR5NK1rA/file# [following]

--2023-03-03 11:20:08-- https://uce3a5cdf30abe772e14002d1d64.dl.dropboxusercontent.com/cd/0/inline/B3i5iKbQi6aU9tcS8iIlC10EzPkychJ7mXM5LhvsLRpIhgsvG-GcdOMSTiQyegGdTHJp9MYWNSCW4gFf10eqy1aJlFmlja30sJHom4YmBpxyGi7hD2u1_COSjuwfv7UxexRAam9enmoC_pOETn11f2cPbZ0YMNIFNI1UV_wR5NK1rA/file

Resolving uce3a5cdf30abe772e14002d1d64.dl.dropboxusercontent.com

(uce3a5cdf30abe772e14002d1d64.dl.dropboxusercontent.com)... 162.125.6.15,

2620:100:6020:15::a27d:400f

Connecting to uce3a5cdf30abe772e14002d1d64.dl.dropboxusercontent.com

(uce3a5cdf30abe772e14002d1d64.dl.dropboxusercontent.com)|162.125.6.15|:443...

connected.

HTTP request sent, awaiting response... 302 Found

Location: /cd/0/inline2/B3h4x3qs0xKZugGzLP63VFMA0dGHmOdXS-

ryN75iWiMfQoGfbxLHf67_R7-FzEWA_Yj--Nsfq8qKE2T8Dc5wLAKKEZM4-r0MKBaDHdcbk9ew4AAI_14j5Fd8dG1zwRpe_DKvERoXmKvgs74MxOEtZ5lYnRtpKpX0b_f9EknAKihz8RvcW26a4ITSJlzk2_Qc1ofYia8EswnqOaG0DrpdC6DomVn6T_1AiVBdxEyOd_4v2DihwFzwWVeMJZ50Zb8byAeZha-jGcL73f00SZAQ77ic0zFhz4GhEITlpNG4_6dH-

alu_DFbjL0rEWS1onthe4qIw3RBEGX9tSmj7LNFyrtzxBkqirOfB7abrTe_8_MU_IeR0-vCW-

Re5kUNxwVRZeou9fk2NMDEODPQfwfJvDnSevPq2LP0U1OHB6jauNLCA/file [following]

--2023-03-03 11:20:08-- https://uce3a5cdf30abe772e14002d1d64.dl.dropboxusercontent.com/cd/0/inline2/B3h4x3qs0xKZugGzLP63VFMA0dGHmOdXS-

ryN75iWiMfQoGfbxLHf67_R7-FzEWA_Yj--Nsfq8qKE2T8Dc5wLAKKEZM4-r0MKBaDHdcbk9ew4AAI_14j5Fd8dG1zwRpe_DKvERoXmKvgs74MxOEtZ5lYnRtpKpX0b_f9EknAKihz8RvcW26a4ITSJlzk2_Qc1ofYia8EswnqOaG0DrpdC6DomVn6T_1AiVBdxEyOd_4v2DihwFzwWVeMJZ50Zb8byAeZha-jGcL73f00SZAQ77ic0zFhz4GhEITlpNG4_6dH-

alu_DFbjL0rEWS1onthe4qIw3RBEGX9tSmj7LNFyrtzxBkqirOfB7abrTe_8_MU_IeR0-vCW-

Re5kUNxwVRZeou9fk2NMDEODPQfwfJvDnSevPq2LP0U1OHB6jauNLCA/file

Reusing existing connection to

uce3a5cdf30abe772e14002d1d64.dl.dropboxusercontent.com:443.

```
HTTP request sent, awaiting response... 200 OK
Length: 9251149 (8.8M) [application/octet-stream]
Saving to: 'Task01_BrainTumour_2D.tar.gz.2'
```

```
Task01_BrainTumour_ 100%[=====>] 8.82M --.-KB/s in 0.04s
```

```
2023-03-03 11:20:08 (227 MB/s) - 'Task01_BrainTumour_2D.tar.gz.2' saved
[9251149/9251149]
```

1.5 Visualise a random set of 4 training images along with their label maps.

Suggested colour map for brain MR image:

```
cmap = 'gray'
```

Suggested colour map for segmentation map:

```
cmap = colors.ListedColormap(['black', 'green', 'blue', 'red'])
```

```
[3]: ### Insert your code ###

rng = np.random.default_rng(seed=256)

# Load the images and labels.
images = os.listdir(training_images_path)
labels = os.listdir(training_labels_path)
num_training_images = len(images)

# Get random indexes.
indexes = rng.integers(1, num_training_images + 1, size=4)

# Get the random images.
images = [images[index] for index in indexes]
labels = [labels[index] for index in indexes]

fig, axes = plt.subplots(nrows=4, ncols=2, figsize=(12, 16))

for i in range(4):
    image_name = images[i]
    label_name = labels[i]

    image = plt.imread(training_images_path + image_name)
    label = plt.imread(training_labels_path + label_name)

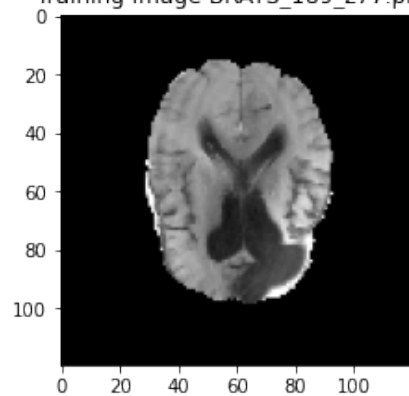
    # Plot the image.
    axes[i, 0].imshow(image, cmap=image_color_map)
    axes[i, 0].set_title("Training image " + image_name)
```

```
# Plot the label.
axes[i, 1].imshow(label, cmap=label_color_map)
axes[i, 1].set_title("Training label " + label_name)

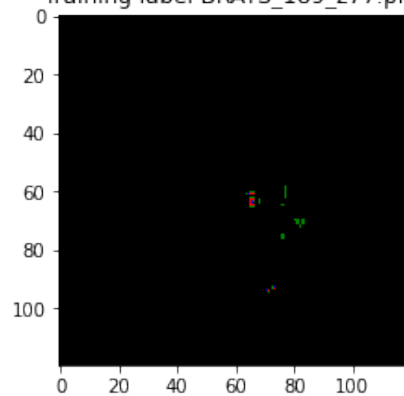
plt.show()

### End of your code ###
```

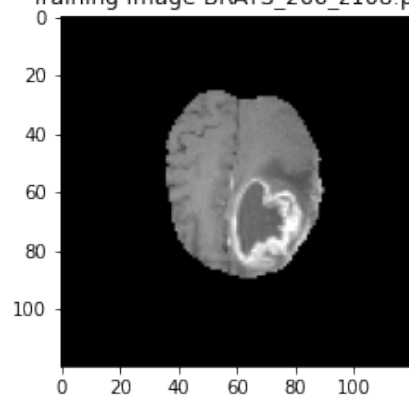
Training image BRATS_189_z77.png



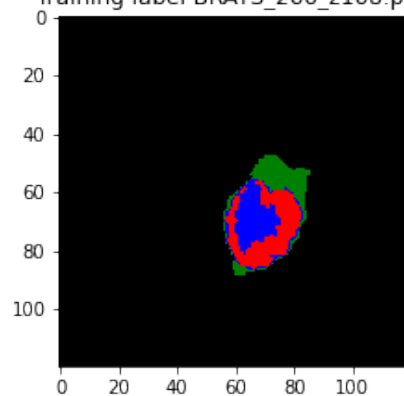
Training label BRATS_189_z77.png



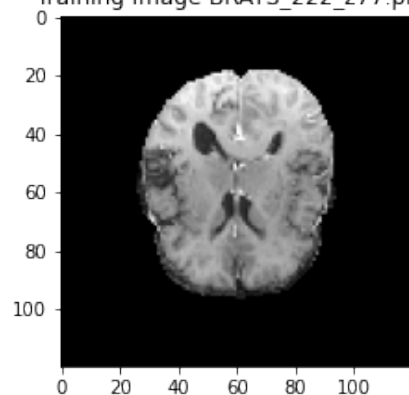
Training image BRATS_266_z108.png



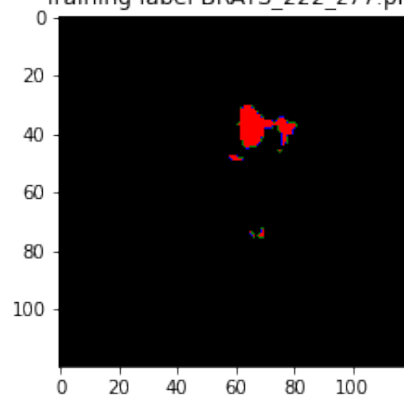
Training label BRATS_266_z108.png



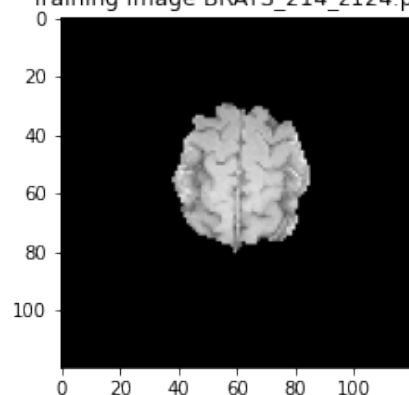
Training image BRATS_222_z77.png



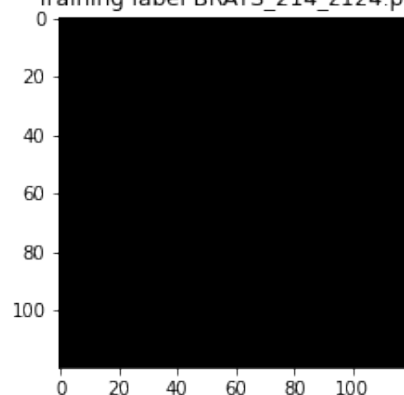
Training label BRATS_222_z77.png



Training image BRATS_214_z124.png



Training label BRATS_214_z124.png



1.6 2. Implement a dataset class.

It can read the imaging dataset and get items, pairs of images and label maps, as training batches.

```
[4]: def normalise_intensity(image, thres_roi=1.0):  
    """ Normalise the image intensity by the mean and standard deviation """  
    # ROI defines the image foreground  
    val_l = np.percentile(image, thres_roi)  
    roi = (image >= val_l)  
    mu, sigma = np.mean(image[roi]), np.std(image[roi])  
    eps = 1e-6  
    image2 = (image - mu) / (sigma + eps)  
    return image2  
  
class BrainImageSet(Dataset):  
    """ Brain image set """  
  
    def __init__(self, image_path, label_path='', deploy=False):  
        self.image_path = image_path  
        self.deploy = deploy  
        self.images = []  
        self.labels = []  
  
        image_names = sorted(os.listdir(image_path))  
        for image_name in image_names:  
            # Read the image  
            image = imageio.imread(os.path.join(image_path, image_name))  
            self.images += [image]  
  
            # Read the label map  
            if not self.deploy:  
                label_name = os.path.join(label_path, image_name)  
                label = imageio.imread(label_name)  
                self.labels += [label]  
  
    def __len__(self):  
        return len(self.images)  
  
    def __getitem__(self, idx):  
        # Get an image and perform intensity normalisation  
        # Dimension: XY  
        image = normalise_intensity(self.images[idx])  
  
        # Get its label map
```

```

        # Dimension: XY
        label = self.labels[idx]
        return image, label

    def get_random_batch(self, batch_size):
        # Get a batch of paired images and label maps
        # Dimension of images: NCXY
        # Dimension of labels: NXY
        images, labels = [], []

        ### Insert your code ###

        # Get random image/label indexes.
        images_indexes = np.random.choice(
            range(0, self.__len__()),
            size=batch_size,
            replace=False,
        )

        # Get the random pairs.
        image_label_pairs = [self.__getitem__(index) for index in
↪ images_indexes]

        # Unzip the images and labels.
        images, labels = map(list, zip(*image_label_pairs))

        images = np.stack([image[np.newaxis, :] for image in images])
        labels = np.array(labels)

        ### End of your code ###

        return images, labels

```

1.7 3. Build a U-net architecture.

You will implement a U-net architecture. If you are not familiar with U-net, please read this paper:

[1] Olaf Ronneberger et al. [U-Net: Convolutional networks for biomedical image segmentation](#). MICCAI, 2015.

For the first convolutional layer, you can start with 16 filters. We have implemented the encoder path. Please complete the decoder path.

```

[5]: """ U-net """

class UNet(nn.Module):
    def __init__(self, input_channel=1, output_channel=1, num_filter=16):

```

```

super(UNet, self).__init__()

# BatchNorm: by default during training this layer keeps running
↳ estimates
# of its computed mean and variance, which are then used for
↳ normalization
# during evaluation.

# Encoder path
n = num_filter # 16
self.conv1 = nn.Sequential(
    nn.Conv2d(input_channel, n, kernel_size=3, padding=1),
    nn.BatchNorm2d(n),
    nn.ReLU(),
    nn.Conv2d(n, n, kernel_size=3, padding=1),
    nn.BatchNorm2d(n),
    nn.ReLU()
)

n *= 2 # 32
self.conv2 = nn.Sequential(
    nn.Conv2d(int(n / 2), n, kernel_size=3, stride=2, padding=1),
    nn.BatchNorm2d(n),
    nn.ReLU(),
    nn.Conv2d(n, n, kernel_size=3, padding=1),
    nn.BatchNorm2d(n),
    nn.ReLU()
)

n *= 2 # 64
self.conv3 = nn.Sequential(
    nn.Conv2d(int(n / 2), n, kernel_size=3, stride=2, padding=1),
    nn.BatchNorm2d(n),
    nn.ReLU(),
    nn.Conv2d(n, n, kernel_size=3, padding=1),
    nn.BatchNorm2d(n),
    nn.ReLU()
)

n *= 2 # 128
self.conv4 = nn.Sequential(
    nn.Conv2d(int(n / 2), n, kernel_size=3, stride=2, padding=1),
    nn.BatchNorm2d(n),
    nn.ReLU(),
    nn.Conv2d(n, n, kernel_size=3, padding=1),
    nn.BatchNorm2d(n),
    nn.ReLU()
)

```

```

    )

    # Decoder path
    ### Insert your code ###

    # n = 128
    self.up_conv1 = nn.ConvTranspose2d(n, int(n / 2), kernel_size=2, ↵
↪stride=2, padding=0)
    n = n // 2 # 64
    self.conv5 = nn.Sequential(
        nn.Conv2d(n * 2, n, kernel_size=3, padding=1),
        nn.BatchNorm2d(n),
        nn.ReLU(),
        nn.Conv2d(n, n, kernel_size=3, padding=1),
        nn.BatchNorm2d(n),
        nn.ReLU()
    )

    # n = 64
    self.up_conv2 = nn.ConvTranspose2d(n, int(n / 2), kernel_size=2, ↵
↪stride=2, padding=0)
    n = n // 2 # 32
    self.conv6 = nn.Sequential(
        nn.Conv2d(n * 2, n, kernel_size=3, padding=1),
        nn.BatchNorm2d(n),
        nn.ReLU(),
        nn.Conv2d(n, n, kernel_size=3, padding=1),
        nn.BatchNorm2d(n),
        nn.ReLU()
    )

    # n = 32
    self.up_conv3 = nn.ConvTranspose2d(n, int(n / 2), kernel_size=2, ↵
↪stride=2, padding=0)
    n = n // 2 # 16
    self.conv7 = nn.Sequential(
        nn.Conv2d(n * 2, n, kernel_size=3, padding=1),
        nn.BatchNorm2d(n),
        nn.ReLU(),
        nn.Conv2d(n, n, kernel_size=3, padding=1),
        nn.BatchNorm2d(n),
        nn.ReLU()
    )

    # n = 16
    self.output_conv = nn.Conv2d(n, output_channel, kernel_size=1)

```

```

    ### End of your code ###

def forward(self, x):
    # Use the convolutional operators defined above to build the U-net
    # The encoder part is already done for you.
    # You need to complete the decoder part.
    # Encoder
    x = self.conv1(x)
    conv1_skip = x

    x = self.conv2(x)
    conv2_skip = x

    x = self.conv3(x)
    conv3_skip = x

    x = self.conv4(x)

    # Decoder
    ### Insert your code ###

    x = self.up_conv1(x)
    x = torch.cat((x, conv3_skip), dim=1)
    x = self.conv5(x)

    x = self.up_conv2(x)
    x = torch.cat((x, conv2_skip), dim=1)
    x = self.conv6(x)

    x = self.up_conv3(x)
    x = torch.cat((x, conv1_skip), dim=1)
    x = self.conv7(x)

    x = self.output_conv(x)

    ### End of your code ###

    return x

```

1.8 4. Train the segmentation model.

```

[6]: # CUDA device
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print('Device: {0}'.format(device))

# Build the model
num_class = 4

```

```

model = UNet(input_channel=1, output_channel=num_class, num_filter=16)
model = model.to(device)
params = list(model.parameters())

model_dir = 'saved_models'
if not os.path.exists(model_dir):
    os.makedirs(model_dir)

# Optimizer
optimizer = optim.Adam(params, lr=1e-3)

# Segmentation loss
criterion = nn.CrossEntropyLoss()

# Datasets
train_set = BrainImageSet('Task01_BrainTumour_2D/training_images',
    ↪ 'Task01_BrainTumour_2D/training_labels')
test_set = BrainImageSet('Task01_BrainTumour_2D/test_images',
    ↪ 'Task01_BrainTumour_2D/test_labels')

# Train the model
# Note: when you debug the model, you may reduce the number of iterations or
    ↪ batch size to save time.
num_iter = 10000
train_batch_size = 16
eval_batch_size = 16
### My code ###
iteration_losses = []
training_losses = []
test_losses = []
### End of my code ###
start = time.time()
for it in range(1, 1 + num_iter):
    # Set the modules in training mode, which will have effects on certain
    ↪ modules, e.g. dropout or batchnorm.
    start_iter = time.time()
    model.train()

    # Get a batch of images and labels
    images, labels = train_set.get_random_batch(train_batch_size)
    images, labels = torch.from_numpy(images), torch.from_numpy(labels)
    images, labels = images.to(device, dtype=torch.float32), labels.to(device,
    ↪ dtype=torch.long)
    logits = model(images)

    # Perform optimisation and print out the training loss
    ### Insert your code ###

```

```

optimizer.zero_grad() # set the gradients to zero
loss = criterion(logits, labels) # compute the loss
loss.backward() # backward pass
optimizer.step() # update the weights

# Print the loss at every 100th iteration and save it.
if it % 100 == 0:
    iteration_losses.append(it)
    training_losses.append(loss.item())
    print(f"Iteration {it}: Training loss = {loss.item():.4f}")

### End of your code ###

# Evaluate
if it % 100 == 0:
    model.eval()
    # Disabling gradient calculation during reference to reduce memory
    ↪consumption
    with torch.no_grad():
        # Evaluate on a batch of test images and print out the test loss
        ### Insert your code ###

        test_images, test_labels = test_set.
        ↪get_random_batch(eval_batch_size)
        test_images, test_labels = torch.from_numpy(test_images), torch.
        ↪from_numpy(test_labels)
        test_images, test_labels = test_images.to(device, dtype=torch.
        ↪float32), test_labels.to(device,

        ↪dtype=torch.long)

        logits = model(test_images)
        loss = criterion(logits, test_labels)

        # Print the test loss and save it.
        test_losses.append(loss.item())
        print(f"Iteration {it}: Test loss = {loss.item():.4f}")

    ### End of your code ###

# Save the model
if it % 5000 == 0:
    torch.save(model.state_dict(), os.path.join(model_dir, 'model_{0}.pt'.
    ↪format(it)))
print('Training took {:.3f}s in total.'.format(time.time() - start))

```

Device: cuda
Iteration 100: Training loss = 0.4663
Iteration 100: Test loss = 0.4786
Iteration 200: Training loss = 0.1952
Iteration 200: Test loss = 0.2108
Iteration 300: Training loss = 0.0933
Iteration 300: Test loss = 0.1473
Iteration 400: Training loss = 0.0914
Iteration 400: Test loss = 0.1007
Iteration 500: Training loss = 0.0688
Iteration 500: Test loss = 0.1240
Iteration 600: Training loss = 0.0610
Iteration 600: Test loss = 0.0834
Iteration 700: Training loss = 0.0518
Iteration 700: Test loss = 0.0342
Iteration 800: Training loss = 0.0431
Iteration 800: Test loss = 0.0845
Iteration 900: Training loss = 0.0318
Iteration 900: Test loss = 0.0825
Iteration 1000: Training loss = 0.0461
Iteration 1000: Test loss = 0.0445
Iteration 1100: Training loss = 0.0593
Iteration 1100: Test loss = 0.0614
Iteration 1200: Training loss = 0.0336
Iteration 1200: Test loss = 0.0450
Iteration 1300: Training loss = 0.0308
Iteration 1300: Test loss = 0.0413
Iteration 1400: Training loss = 0.0250
Iteration 1400: Test loss = 0.0588
Iteration 1500: Training loss = 0.0275
Iteration 1500: Test loss = 0.0603
Iteration 1600: Training loss = 0.0580
Iteration 1600: Test loss = 0.0410
Iteration 1700: Training loss = 0.0301
Iteration 1700: Test loss = 0.0543
Iteration 1800: Training loss = 0.0464
Iteration 1800: Test loss = 0.0266
Iteration 1900: Training loss = 0.0321
Iteration 1900: Test loss = 0.0285
Iteration 2000: Training loss = 0.0267
Iteration 2000: Test loss = 0.0316
Iteration 2100: Training loss = 0.0306
Iteration 2100: Test loss = 0.0287
Iteration 2200: Training loss = 0.0196
Iteration 2200: Test loss = 0.0381
Iteration 2300: Training loss = 0.0154
Iteration 2300: Test loss = 0.0474
Iteration 2400: Training loss = 0.0303

Iteration 2400: Test loss = 0.0380
Iteration 2500: Training loss = 0.0253
Iteration 2500: Test loss = 0.0477
Iteration 2600: Training loss = 0.0254
Iteration 2600: Test loss = 0.0296
Iteration 2700: Training loss = 0.0171
Iteration 2700: Test loss = 0.0822
Iteration 2800: Training loss = 0.0235
Iteration 2800: Test loss = 0.0513
Iteration 2900: Training loss = 0.0159
Iteration 2900: Test loss = 0.0333
Iteration 3000: Training loss = 0.0243
Iteration 3000: Test loss = 0.0268
Iteration 3100: Training loss = 0.0145
Iteration 3100: Test loss = 0.0359
Iteration 3200: Training loss = 0.0196
Iteration 3200: Test loss = 0.0182
Iteration 3300: Training loss = 0.0127
Iteration 3300: Test loss = 0.0594
Iteration 3400: Training loss = 0.0130
Iteration 3400: Test loss = 0.0426
Iteration 3500: Training loss = 0.0183
Iteration 3500: Test loss = 0.0352
Iteration 3600: Training loss = 0.0119
Iteration 3600: Test loss = 0.0436
Iteration 3700: Training loss = 0.0115
Iteration 3700: Test loss = 0.0288
Iteration 3800: Training loss = 0.0195
Iteration 3800: Test loss = 0.0328
Iteration 3900: Training loss = 0.0160
Iteration 3900: Test loss = 0.0254
Iteration 4000: Training loss = 0.0129
Iteration 4000: Test loss = 0.0420
Iteration 4100: Training loss = 0.0144
Iteration 4100: Test loss = 0.0468
Iteration 4200: Training loss = 0.0234
Iteration 4200: Test loss = 0.0309
Iteration 4300: Training loss = 0.0137
Iteration 4300: Test loss = 0.0653
Iteration 4400: Training loss = 0.0109
Iteration 4400: Test loss = 0.0457
Iteration 4500: Training loss = 0.0113
Iteration 4500: Test loss = 0.0472
Iteration 4600: Training loss = 0.0203
Iteration 4600: Test loss = 0.0225
Iteration 4700: Training loss = 0.0145
Iteration 4700: Test loss = 0.0237
Iteration 4800: Training loss = 0.0103

Iteration 4800: Test loss = 0.0379
Iteration 4900: Training loss = 0.0159
Iteration 4900: Test loss = 0.0273
Iteration 5000: Training loss = 0.0115
Iteration 5000: Test loss = 0.0545
Iteration 5100: Training loss = 0.0135
Iteration 5100: Test loss = 0.0380
Iteration 5200: Training loss = 0.0102
Iteration 5200: Test loss = 0.0414
Iteration 5300: Training loss = 0.0120
Iteration 5300: Test loss = 0.0632
Iteration 5400: Training loss = 0.0086
Iteration 5400: Test loss = 0.0611
Iteration 5500: Training loss = 0.0102
Iteration 5500: Test loss = 0.0392
Iteration 5600: Training loss = 0.0165
Iteration 5600: Test loss = 0.0397
Iteration 5700: Training loss = 0.0135
Iteration 5700: Test loss = 0.0688
Iteration 5800: Training loss = 0.0125
Iteration 5800: Test loss = 0.0460
Iteration 5900: Training loss = 0.0089
Iteration 5900: Test loss = 0.0499
Iteration 6000: Training loss = 0.0116
Iteration 6000: Test loss = 0.0562
Iteration 6100: Training loss = 0.0141
Iteration 6100: Test loss = 0.0887
Iteration 6200: Training loss = 0.0105
Iteration 6200: Test loss = 0.0414
Iteration 6300: Training loss = 0.0133
Iteration 6300: Test loss = 0.0559
Iteration 6400: Training loss = 0.0113
Iteration 6400: Test loss = 0.0689
Iteration 6500: Training loss = 0.0087
Iteration 6500: Test loss = 0.0448
Iteration 6600: Training loss = 0.0186
Iteration 6600: Test loss = 0.0297
Iteration 6700: Training loss = 0.0057
Iteration 6700: Test loss = 0.0403
Iteration 6800: Training loss = 0.0086
Iteration 6800: Test loss = 0.0496
Iteration 6900: Training loss = 0.0081
Iteration 6900: Test loss = 0.0468
Iteration 7000: Training loss = 0.0128
Iteration 7000: Test loss = 0.0708
Iteration 7100: Training loss = 0.0098
Iteration 7100: Test loss = 0.0345
Iteration 7200: Training loss = 0.0088

Iteration 7200: Test loss = 0.0471
Iteration 7300: Training loss = 0.0095
Iteration 7300: Test loss = 0.0443
Iteration 7400: Training loss = 0.0128
Iteration 7400: Test loss = 0.0230
Iteration 7500: Training loss = 0.0099
Iteration 7500: Test loss = 0.0489
Iteration 7600: Training loss = 0.0049
Iteration 7600: Test loss = 0.0473
Iteration 7700: Training loss = 0.0061
Iteration 7700: Test loss = 0.0517
Iteration 7800: Training loss = 0.0095
Iteration 7800: Test loss = 0.0498
Iteration 7900: Training loss = 0.0150
Iteration 7900: Test loss = 0.0325
Iteration 8000: Training loss = 0.0141
Iteration 8000: Test loss = 0.0355
Iteration 8100: Training loss = 0.0072
Iteration 8100: Test loss = 0.0296
Iteration 8200: Training loss = 0.0065
Iteration 8200: Test loss = 0.0387
Iteration 8300: Training loss = 0.0130
Iteration 8300: Test loss = 0.0454
Iteration 8400: Training loss = 0.0101
Iteration 8400: Test loss = 0.0194
Iteration 8500: Training loss = 0.0107
Iteration 8500: Test loss = 0.0321
Iteration 8600: Training loss = 0.0075
Iteration 8600: Test loss = 0.0721
Iteration 8700: Training loss = 0.0131
Iteration 8700: Test loss = 0.0624
Iteration 8800: Training loss = 0.0075
Iteration 8800: Test loss = 0.0287
Iteration 8900: Training loss = 0.0067
Iteration 8900: Test loss = 0.0254
Iteration 9000: Training loss = 0.0112
Iteration 9000: Test loss = 0.0441
Iteration 9100: Training loss = 0.0078
Iteration 9100: Test loss = 0.0497
Iteration 9200: Training loss = 0.0076
Iteration 9200: Test loss = 0.0231
Iteration 9300: Training loss = 0.0093
Iteration 9300: Test loss = 0.0415
Iteration 9400: Training loss = 0.0068
Iteration 9400: Test loss = 0.0544
Iteration 9500: Training loss = 0.0103
Iteration 9500: Test loss = 0.0670
Iteration 9600: Training loss = 0.0105

```

Iteration 9600: Test loss = 0.0520
Iteration 9700: Training loss = 0.0090
Iteration 9700: Test loss = 0.0706
Iteration 9800: Training loss = 0.0076
Iteration 9800: Test loss = 0.0213
Iteration 9900: Training loss = 0.0070
Iteration 9900: Test loss = 0.0371
Iteration 10000: Training loss = 0.0100
Iteration 10000: Test loss = 0.0562
Training took 356.196s in total.

```

1.9 5. Deploy the trained model to a random set of 4 test images and visualise the automated segmentation.

You can show the images as a 4 x 3 panel. Each row shows one example, with the 3 columns being the test image, automated segmentation and ground truth segmentation.

```

[7]: ### Insert your code ###

# Load the model.
model.load_state_dict(torch.load(os.path.join(model_dir, 'model_10000.pt')))

images, labels = test_set.get_random_batch(4)
images, labels = torch.from_numpy(images), torch.from_numpy(labels)
images, labels = images.to(device, dtype=torch.float32), labels.to(device,
↳dtype=torch.long)
# Get the model's output labels.
with torch.no_grad():
    predicted_labels = model(images)

fig, axs = plt.subplots(nrows=4, ncols=3, figsize=(20, 30))

for i in range(4):
    test_image = images[i]
    test_label = labels[i]
    output_label = predicted_labels[i].squeeze(dim=0).argmax(dim=0)

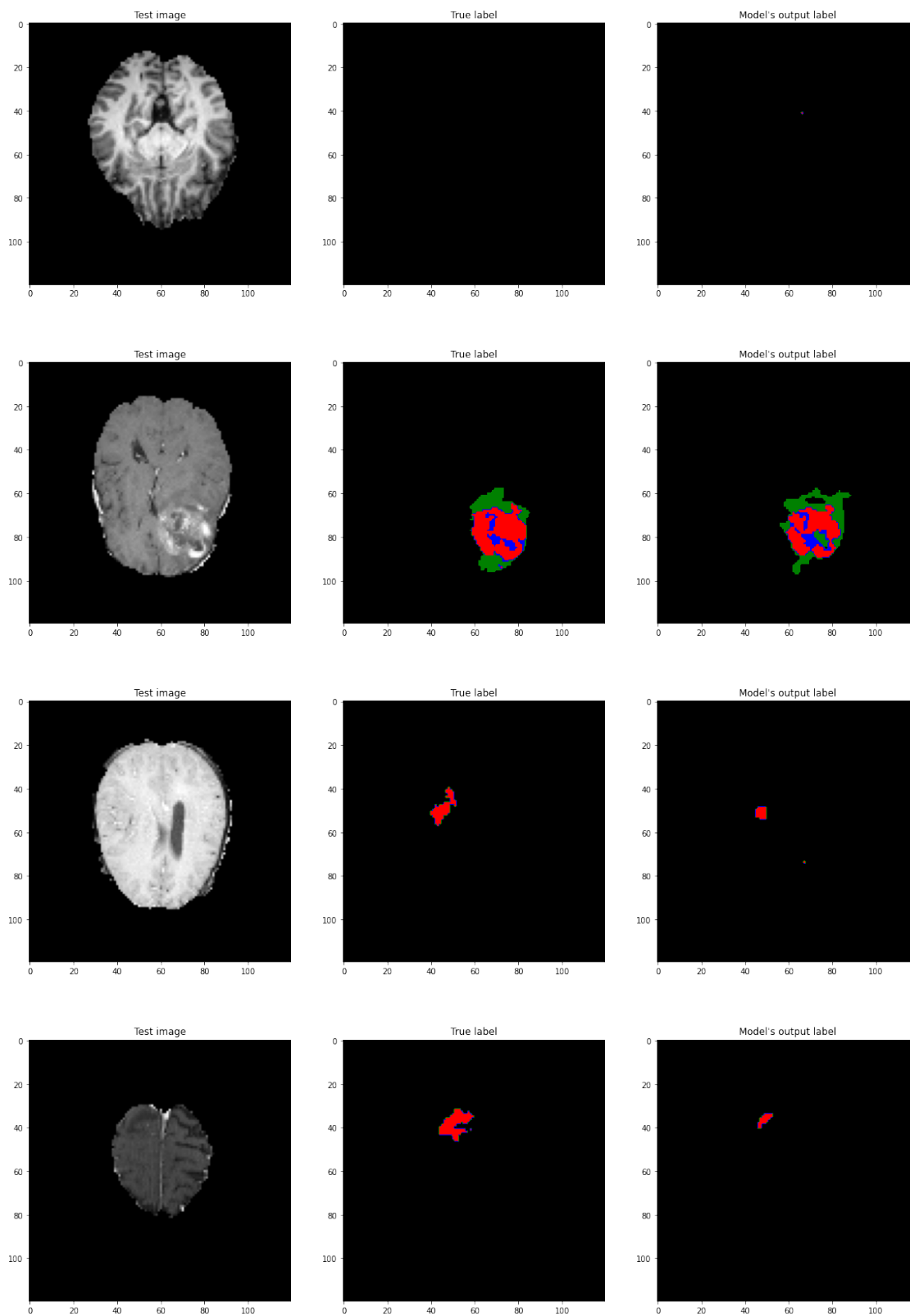
    # Plot the image.
    axs[i, 0].imshow(test_image.cpu().numpy().squeeze(), cmap=image_color_map)
    axs[i, 0].set_title("Test image")

    # Plot the true label.
    axs[i, 1].imshow(test_label.cpu(), cmap=label_color_map)
    axs[i, 1].set_title("True label")

    # Plot the model's output label.
    axs[i, 2].imshow(output_label.cpu().numpy(), cmap=label_color_map)
    axs[i, 2].set_title("Model's output label")

```

```
### End of your code ###
```



1.10 6. Discussion. Does your trained model work well? How would you improve this model so it can be deployed to the real clinic?

```
[8]: plt.figure(figsize=(16, 16))
font_size = 10

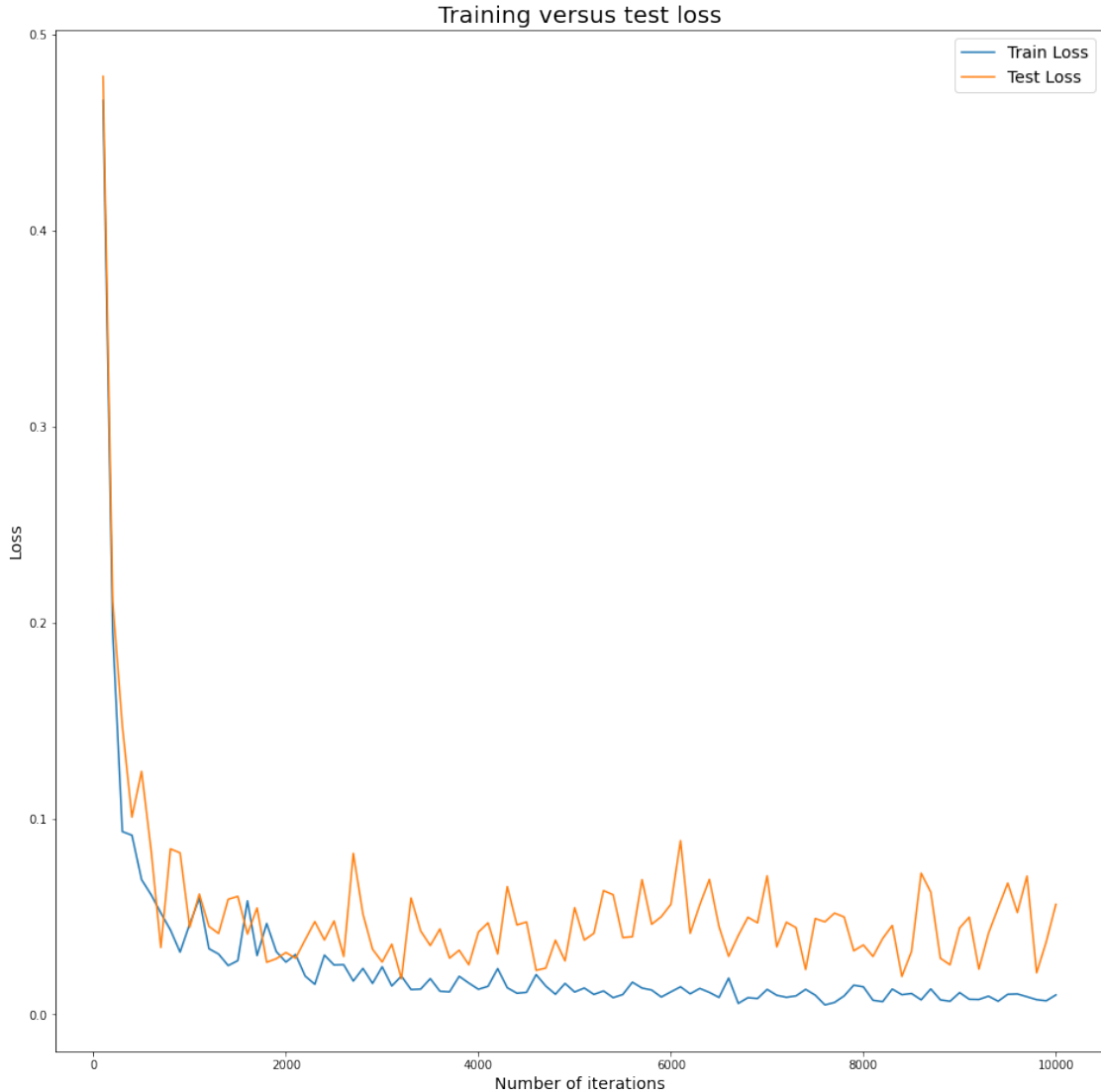
plt.title("Training versus test loss", fontsize=2 * font_size)

plt.xlabel("Number of iterations", fontsize=font_size + 4)
plt.ylabel("Loss", fontsize=font_size + 4)
plt.xticks(size=font_size)
plt.yticks(size=font_size)

plt.plot(iteration_losses, training_losses, label="Train Loss")
plt.plot(iteration_losses, test_losses, label="Test Loss")

plt.legend(fontsize=font_size + 4)

plt.show()
```



After 10000 iterations, the model seems to perform reasonably well, with a cross-entropy loss at $\approx 0.005 - 0.01$ on the training set and ≈ 0.05 on test set.

By looking at the previous plot, we see that after the 2000th iteration the model does not improve significantly and the test loss starts fluctuating. Although this may be due to randomness, it's possible that the model is overfitting.

In order to improve the model, we could:

- Increase the size and quality of the training dataset. By looking at the images, we see that they are 120×120 , which is a really low resolution. It may not capture all the information needed in order to make accurate predictions. By getting more data and increasing its quality, the model would be able to learn better features and improve its accuracy.
- In order to avoid overfitting, we could:
 1. Tune the hyperparameters (batch size, number of iterations) using cross-validation.

2. Add early stopping.
 3. Since the model has a high number of parameters, we could add some regularisation (L1 or L2).
- Change the loss function: this can have a significant impact on the model's performance.
 - We can try to make some changes to the architecture. For example, we could add or remove convolutional layers, modify the depth of the network, use skip connections, change kernel size or the pooling methods.
 - We can use ensemble models. By training multiple U-Net models (possibly on different datasets), with different loss functions or architectures, we can obtain a model that performs better than an individual one.

If we want to deploy it to a real clinic, we need to make sure that:

- We take appropriate measures in order to ensure the patients' privacy and confidentiality.
- As the model is intended for medical purposes, we need to conduct rigorous testing before deploying it.